# Test suite and process evidence

**CSC322 Group E — CalcEngine**

```
dotnet test                                        # everything, ~5 s
dotnet test --filter "Category!=Performance"       # without the timing guards, ~2 s
dotnet test --filter "Category=Performance"        # the two published targets only
```

382 tests, all green.

---

## 1. How the suite is organised

| Area | Tests | What it pins |
|---|---|---|
| `Model/` | 77 | The three foundational ADTs: bijective base-26 columns, normalising range corners, the tagged-union discipline of `CellValue`, invariant-culture display |
| `Parsing/` | 79 | Tree shape for every construct, the Excel precedence table, the print/parse round trip, and the exact wording and column of every syntax error |
| `Evaluation/` | 95 | Coercions, leftmost-error propagation, the eight required functions and the supporting library, registry extension |
| `Workbooks/` | 51 | Content classification, reactive propagation, edge removal, cross-sheet references, circular references |
| `Commands/` | 17 | Undo, redo, bounded history, batch and composite operations, suspension |
| `Features/` | 57 | Find & Replace and duplicate detection, including every way each can refuse |
| `Performance/` | 3 | The two published targets and the locality claim behind them |
| `ReadmeExampleTests` | 3 | Every value quoted in README.md and in the grammar's error catalogue, executed |
| *Total* | **382** | |

---

## 2. Tests we would point a marker at

Coverage numbers say how much code ran. These say what the engine promises.

`OnlyAffectedCellsAreRecomputed` asserts on `CalculationResult.CellsEvaluated`, not on cell values. A workbook that recalculated everything on every edit would produce identical values and fail this test. It is the reactive claim, made falsifiable.

`WritingIntoAPreviouslyEmptyCellOfARangeStillTriggersIt` is the single reason `RangeDependencyIndex` exists rather than an expanded edge list. It is the case a plausible wrong design silently fails.

`If_DoesNotEvaluateTheBranchItDoesNotTake` evaluates `=IF(A1=0,"n/a",B1/A1)` with `A1 = 0`. An eager function interface returns `#DIV/0!` here — for exactly the input the formula was written to guard against.

`TheCycleIsReportedTheSameWayWhicheverCellClosedIt` builds the same three-cell ring in two different orders and requires identical output, because a diagnostic that changes between runs is not one you can act on.

`ALongCycleDoesNotOverflowTheStack` and `ADeepChainDoesNotOverflowTheStack` use 20,000 cells. `StackOverflowException` cannot be caught in .NET, so a recursive traversal does not fail the operation, it kills the host process.

`ANumberIsNotTheSameAsTheTextThatLooksLikeIt` is duplicate detection's equivalent: an imported column of text marks must not appear to duplicate a typed one.

`ReplaceAll_InTextLiteralsOnlyLeavesReferencesUntouched` and `ReplaceAll_InWholeContentModeWouldHaveBrokenThatFormula` sit next to each other on purpose. The second documents precisely how the default mode rewrites `=SUM(B2:B9)`, so the difference between the two modes is behaviour under test rather than a footnote.

`SyntaxErrorMessageTests` pins the exact string of every parser message. Error messages that drift are error messages nobody trusts.

`ReadmeExampleTests` executes the example in `README.md` and the error catalogue in `grammar.md` §6. Documentation that is not executed is documentation that is wrong by the end of term — this suite already caught one inaccurate README claim (`Changes[0]` is the edited cell, not its dependent) and one badly worded diagnostic (`ToSentence()` omitted a "which").

---

## 3. Test-first, and what the history shows

`git log --oneline` reads in pairs. A `test(...)` commit lands first, described in its own message as red, containing tests for types that do not exist yet; the `feat(...)` commit that follows makes them pass and says how many are green.

```
test(model): specification tests for CellAddress, CellRange and CellValue
feat(model): CellAddress, CellRange, CellValue and ErrorValue ADTs
test(parsing): expression tree and parser specification tests
feat(parsing): expression tree ADT, AST builder and diagnostic parser
test(evaluation): operator, coercion and function-library specification tests
feat(evaluation): Interpreter evaluator, coercions and function library
test(workbook): recalculation and circular-reference specification tests
feat(workbook): dependency graph, reactive recalculation and cycle reporting
test(commands): undo/redo specification tests
feat(commands): undo/redo history built on the Command pattern
test(find-replace): specification tests for assigned feature 1
feat(find-replace): assigned feature 1, with formula-safe replacement
test(duplicates): specification tests for assigned feature 2
feat(duplicates): assigned feature 2, detection and removal
```

The red commits do not build. That is the point: they are the specification, committed before the implementation, and the following commit is the evidence that the specification was met rather than adjusted.

Where a red commit's expectation was itself wrong, the green commit says so rather than quietly editing it. Three times:

- the unmatched `)` in `=SUM(A1))` is at column 9, not 8;
- the fully parenthesised printer brackets a percent node like every other operator;
- `RecalculateAll` on a sheet with one literal and two formulas evaluates two cells, not three.

---

## 4. What the tests did not catch

Three real defects reached the repository and were found by *running* the system, not by testing it. They are recorded here because a test suite's honest measure includes what got past it.

`Workbook.AutomaticCalculation` **could not be switched back on.** The property was initialised but every edit consulted the constructor option instead, so a workbook created for a bulk import stayed in manual mode forever. Every unit test passed, because no unit test loaded a workbook the way a real client would. The benchmark harness was the first code that did, and it failed on its first run. Now guarded by `CalculationCanBeSwitchedBackOnAfterABulkLoad`.

**The grid deleted** `A1` **on the first click.** `default(CellAddress)` is `A1`, so the component's "has the selection moved?" test was false on the first render, the edit buffer was never seeded, and the first click elsewhere committed an empty string over the header. Found by looking at a screenshot from `tools/gui-smoke.js`.

**The selected cell went stale after a replacement.** The same "has the selection moved?" test decided when to re-read the workbook, so a Replace All — or an undo, or a commit typed into the formula bar — that rewrote the cell the user was sitting on left the grid showing the old text while the

formula bar showed the new one. The data was always correct; only the one cell under the selection lied about it. Reported by a user driving the GUI by hand, which no automated check we had was doing. The editor now re-seeds on the right key — "is the buffer still what the workbook holds?" — and `tools/gui-smoke.js` §8 compares each cell's rendered text against its tooltip, which is read straight from the workbook.

The lesson we took: unit tests prove the parts, running the whole thing proves the seams — and the seam nobody exercised was the one where two views of the same cell can disagree. All three defects are in the client or in a code path no test drove like a real client would; none is in `CalcEngine.Core`.

---

## 5. Running the performance targets

See `benchmarks.md`. In short:

```
dotnet run -c Release --project benchmarks/CalcEngine.Benchmarks
```

exits non-zero if either published target is missed, so it can gate CI. The same targets are asserted by `PerformanceTargetTests` in the ordinary Debug test configuration, where they still pass with two orders of magnitude to spare.

## 6. Running the GUI end to end

```
dotnet run --project src/CalcEngine.Gui -c Release
# then, in another terminal:
npm install playwright && SHOTS=./shots node tools/gui-smoke.js
```

The script loads the sample results sheet, edits a mark and checks that the total, the grade and the class average all move, scans for duplicates, inserts a circular reference and reads the reported cycle back off the screen, types a malformed formula and reads the parser's message off the status bar, then replaces text and undoes it. An empty `errors` array in its JSON report means no failed requests and no browser exceptions.